



МИНИСТЕРСТВО НАУКИ
И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное
образовательное учреждение высшего образования
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»



**НГТУ
НЭТИ** | **Факультет прикладной
математики и информатики**

Кафедра теоретической и прикладной информатики
Лабораторная работа № 2
по дисциплине «Методы оптимизации»

МЕТОДЫ СПУСКА

Бригада 9	ГРУШЕВ АНДРЕЙ
Группа ПМ-05	ХАБАРОВА АНАСТАСИЯ
Вариант 9	

Преподаватели	ЛЕМЕШКО БОРИС ЮРЬЕВИЧ
	ФИЛИППОВА ЕЛЕНА ВЛАДИМИРОВНА

Новосибирск, 2023

Цель работы

Ознакомиться с методами поиска минимума функции n переменных в оптимизационных задачах без ограничений.

Задание

1. Реализовать два метода поиска экстремума функции (разного порядка). Включить в реализуемый алгоритм собственную процедуру, реализующую одномерный поиск по направлению.
2. С использованием разработанного программного обеспечения исследовать алгоритмы на квадратичной функции, функции Розенброка и на заданной в соответствии с вариантом тестовой функции, осуществляя спуск из различных исходных точек (не менее двух). Исследовать сходимость алгоритма, фиксируя точность определения минимума/максимума, количество итераций метода и количество вычислений функции в зависимости от задаваемой точности поиска.

Минимизируемая функция: $f(x, y) = 2\exp\left\{-\left(\frac{x-3}{1}\right)^2 - \left(\frac{y-2}{1}\right)^2\right\} + 3\exp\left\{-\left(\frac{x-1}{1}\right)^2 - \left(\frac{y-1}{3}\right)^2\right\}.$

Результаты исследований

Метод Розенброка

Квадратичная функция:

Начальное приближение: (10.0; -8.0)

x_0	y_0	ε	$iters$	$funcs$	x	y	$f(x, y)$
10.000000	-8.000000	1.E-03	4	289	1.000113	0.999995	1.407736E-06
10.000000	-8.000000	1.E-04	4	368	0.999999	1.000011	1.479285E-08
10.000000	-8.000000	1.E-05	4	433	1.000002	1.000002	7.103461E-12
10.000000	-8.000000	1.E-06	4	503	1.000000	1.000000	7.479941E-14
10.000000	-8.000000	1.E-07	4	597	1.000000	1.000000	2.863152E-14

$iter$	x_i	y_i	$f(x_i, y_i)$	s_1	s_2	λ	$ x_i - x_{i-1} $	$ y_i - y_{i-1} $	$ f_i - f_{i-1} $	$angle$
1	-7.910947	-8.000000	8.019802E+01	1.000000	0.000000	-17.910947	1.791095E+01	0.000000E+00	3.240080E+04	134.679320
1	-7.910947	-7.910840	7.940497E+01	0.000000	1.000000	0.089160	1.791095E+01	8.916045E-02	3.240160E+04	134.999612
2	-7.823533	-7.911275	7.862460E+01	-0.999988	0.004978	-0.087415	8.741403E-02	4.351459E-04	7.803779E-01	45.604710
2	-7.823091	-7.822598	7.784696E+01	0.004978	0.999988	0.088678	8.785546E-02	8.824181E-02	1.558009E+00	135.283408
3	0.982803	1.022021	1.540971E-01	0.705554	0.708656	12.480827	8.805894E+00	8.844618E+00	7.769287E+01	0.994950
3	1.002411	1.002499	6.588110E-06	0.708656	-0.705554	0.027669	8.825502E+00	8.825097E+00	7.784696E+01	89.876815
4	1.000010	1.000098	7.784404E-07	0.707123	0.707091	-0.003395	2.400769E-03	2.400658E-03	5.809669E-06	0.003843
4	1.000113	0.999995	1.407736E-06	0.707091	-0.707123	0.000146	2.297606E-03	2.503826E-03	5.180374E-06	89.997933

Начальное приближение: (-2500.0; 6000.0)

x_0	y_0	ε	$iters$	$funcs$	x	y	$f(x, y)$
-2500	6000	1.E-03	5	589	1.000154	1.000112	2.050142E-07
-2500	6000	1.E-04	6	708	1.000005	0.999985	3.975911E-08
-2500	6000	1.E-05	6	815	1.000000	1.000001	7.968012E-12
-2500	6000	1.E-06	6	924	1.000000	1.000000	2.431338E-12
-2500	6000	1.E-07	6	1023	1.000000	1.000000	2.813846E-14

$iter$	x_i	y_i	$f(x_i, y_i)$	s_1	s_2	λ	$ x_i - x_{i-1} $	$ y_i - y_{i-1} $	$ f_i - f_{i-1} $	$angle$
1	5940.604	6000	35631684	1	0	8440.604	8440.604	0	7.2E+09	45.285
1	5940.604	5940.604	35278895	0	1	-59.3962	8440.604	59.39618	7.2E+09	45
2	5882.607	5941.012	34934415	0.999975	-0.00704	-57.9984	57.99693	0.408122	344480.2	45.6862
2	5882.19	5881.773	34588415	-0.00704	-0.99998	59.23999	58.41379	58.8304	690480	135.4012
3	31.05998	-11.0874	178544.1	-0.70459	-0.70961	8304.308	5851.13	5892.861	34409871	115.1514
3	9.888891	9.933723	79.21337	-0.70961	0.70459	29.83461	5872.301	5871.84	34588336	90.07401
4	1.00044	1.04597	0.207297	-0.70713	-0.70708	12.56967	8.888451	8.887752	79.00608	178.7232
4	1.023065	1.023344	0.00054	-0.70708	0.707135	-0.032	8.865827	8.910379	79.21283	89.98993
5	1.000051	1.000214	2.68E-06	-0.70533	-0.70888	0.032628	0.023014	0.023129	0.000537	179.8611
5	1.000154	1.000112	2.05E-07	0.708877	-0.70533	0.000146	0.02291	0.023232	0.00054	89.85518

Функция Розенброка:

Начальное приближение: (10.0; -8.0)

x_0	y_0	ε	$iters$	$funcs$	x	y	$f(x, y)$
10.000000	-8.000000	1.E-03	15	924	1.000244	1.000180	9.537304E-06
10.000000	-8.000000	1.E-04	16	1283	0.999927	0.999824	8.972464E-08
10.000000	-8.000000	1.E-05	16	1631	0.999963	0.999924	1.783150E-09
10.000000	-8.000000	1.E-06	17	2056	1.000000	1.000000	1.637982E-12
10.000000	-8.000000	1.E-07	17	2420	1.000000	1.000000	1.840818E-13

$iter$	x_i	y_i	$f(x_i, y_i)$	s_1	s_2	λ	$ x_i - x_{i-1} $	$ y_i - y_{i-1} $	$ f_i - f_{i-1} $	$angle$
1	0.000449	-8	6400.999	1	0	-9.99955	9.999551	0	1160080	89.99678
1	0.000449	5.53E-05	0.999102	0	1	8.000055	9.999551	8.000055	1166480	82.98336
2	0.015086	-0.01165	0.984174	-0.78085	0.624715	-0.01874	0.014637	0.01171	0.014928	179.0263
2	0.031733	0.009153	0.944176	0.624715	0.780853	0.026647	0.031284	0.009097	0.054925	35.24966
3	0.340115	0.098832	0.463829	0.960222	0.279237	0.321157	0.308382	0.089679	0.480347	0.01171
3	0.336463	0.11139	0.440611	0.279237	-0.96022	-0.01308	0.30473	0.102238	0.503565	92.1031
4	0.372469	0.12347	0.417091	0.948065	0.318078	0.037978	0.036006	0.01208	0.02352	0.206784
4	0.368835	0.134303	0.398671	-0.31808	0.948065	0.011426	0.032372	0.022912	0.04194	88.53878
5	0.487184	0.218069	0.300148	0.816233	0.577723	0.144994	0.118349	0.083766	0.098523	11.17673
5	0.480105	0.22807	0.270882	-0.57772	0.816233	0.012252	0.11127	0.093767	0.127789	99.88092
6	0.554389	0.290669	0.226387	0.764688	0.6444	0.097143	0.074284	0.062599	0.044495	12.45251
6	0.547892	0.298378	0.204728	-0.6444	0.764688	0.010082	0.067787	0.070309	0.066154	101.5483
7	0.637853	0.391685	0.154168	0.694081	0.719897	0.129611	0.08996	0.093306	0.050561	14.49325
7	0.631855	0.397467	0.135845	-0.7199	0.694081	0.008331	0.083963	0.099089	0.068883	103.8741
8	0.703829	0.482408	0.104533	0.646472	0.762938	0.111333	0.071974	0.08494	0.031312	15.29693
8	0.698613	0.486827	0.090986	-0.76294	0.646472	0.006836	0.066759	0.08936	0.044859	104.8533
9	0.772585	0.585841	0.063919	0.5985	0.801123	0.123594	0.073971	0.099014	0.027067	16.06486
9	0.76818	0.589132	0.053834	-0.80112	0.5985	0.005498	0.069566	0.102305	0.037152	105.7521
10	0.83024	0.680399	0.03674	0.562304	0.826931	0.110369	0.062061	0.091267	0.017095	16.44941
10	0.826709	0.6828	0.030072	-0.82693	0.562304	0.004271	0.058529	0.093668	0.023763	106.2305
11	0.883856	0.774256	0.018312	0.529911	0.848053	0.107843	0.057147	0.091456	0.011759	16.78226
11	0.880977	0.776056	0.014167	-0.84805	0.529911	0.003395	0.054268	0.093255	0.015905	106.6236
12	0.931106	0.8622	0.007011	0.502963	0.864308	0.099669	0.05013	0.086144	0.007156	17.00423
12	0.929396	0.863195	0.005019	-0.86431	0.502963	0.001979	0.048419	0.08714	0.009148	106.9188
13	0.960089	0.918432	0.002707	0.485708	0.874121	0.063192	0.030693	0.055237	0.002312	17.21144
13	0.958778	0.919161	0.0017	-0.87412	0.485708	0.0015	0.029381	0.055966	0.003319	107.1496
14	0.989463	0.977611	0.000315	0.464828	0.885401	0.066016	0.030686	0.05845	0.001386	17.64613
14	0.988762	0.977979	0.000137	-0.8854	0.464828	0.000792	0.029985	0.058818	0.001563	107.615
15	1.000114	1.000246	4.69E-08	0.454176	0.890912	0.024994	0.011352	0.022267	0.000137	17.98429

15	1.000244	1.00018	9.54E-06	-0.89091	0.454176	-0.00015	0.011482	0.022201	0.000128	107.9899
----	----------	---------	----------	----------	----------	----------	----------	----------	----------	----------

Начальное приближение: (-2500.0; 6000.0)

x_0	y_0	ε	$iters$	$funcs$	x	y	$f(x, y)$
-2500	6000	1.E-03	2	152	-77.459407	5999.959836	6.155879E+03
-2500	6000	1.E-04	2	169	77.459603	5999.990005	5.846071E+03
-2500	6000	1.E-05	232	29085	0.999999	1.000000	4.380660E-10
-2500	6000	1.E-06	229	34016	1.000000	1.000000	1.595686E-12
-2500	6000	1.E-07	229	39048	1.000000	1.000000	8.102893E-15

$iter$	x_i	y_i	$f(x_i, y_i)$	s_1	s_2	λ	$ x_i - x_{i-1} $	$ y_i - y_{i-1} $	$ f_i - f_{i-1} $	$angle$
1	-77.4596	6000	6155.933	1	0	2422.54	2422.54	0	3.9E+15	90.73964
1	-77.4596	5999.982	6155.901	0	1	-0.01792	2422.54	0.017918	3.9E+15	0.739645
2	-77.4594	5999.982	6155.929	1	-7.40E-06	0.000146	0.000146	1.08E-09	0.027242	90.74007
2	-77.4594	5999.96	6155.879	-7.40E-06	-1	0.022246	0.000146	0.022246	0.022871	179.2599

Функция по варианту:

Начальное приближение: (10.0; -8.0)

x_0	y_0	ε	$iters$	$funcs$	x	y	$f(x, y)$
10.000000	-8.000000	1.E-03	1	77	75.534616	-7.999382	0.000000E+00
10.000000	-8.000000	1.E-04	1	87	62.428655	-7.999938	0.000000E+00
10.000000	-8.000000	1.E-05	1	98	51.943027	-7.999994	0.000000E+00
10.000000	-8.000000	1.E-06	1	111	77.108863	-7.999999	0.000000E+00
10.000000	-8.000000	1.E-07	1	122	63.687091	-8.000000	0.000000E+00

$iter$	x_i	y_i	$f(x_i, y_i)$	s_1	s_2	λ	$ x_i - x_{i-1} $	$ y_i - y_{i-1} $	$ f_i - f_{i-1} $	$angle$
1	75.53462	-8	0	1	0	65.53462	65.53462	0	2.46E-39	6.045755
1	75.53462	-7.99938	0	0	1	0.000618	65.53462	0.000618	2.46E-39	96.04529

Начальное приближение: (-2500.0; 6000.0)

x_0	y_0	ε	$iters$	$funcs$	x	y	$f(x,y)$
-2500	6000	1.E-03	1	24	-2499.999382	6000.000618	0.000000E+00
-2500	6000	1.E-04	1	24	-2499.999938	6000.000062	0.000000E+00
-2500	6000	1.E-05	1	24	-2499.999994	6000.000006	0.000000E+00
-2500	6000	1.E-06	1	24	-2499.999999	6000.000001	0.000000E+00
-2500	6000	1.E-07	1	24	-2500.000000	6000.000000	0.000000E+00

$iter$	x_i	y_i	$f(x_i, y_i)$	s_1	s_2	λ	$ x_i - x_{i-1} $	$ y_i - y_{i-1} $	$ f_i - f_{i-1} $	$angle$
1	-2500	6000	0	1	0	0.000618	0.000618	0	0	112.6199
1	-2500	6000.001	0	0	1	0.000618	0.000618	0.000618	0	22.61986

Метод Бroyдена

Квадратичная функция:

Начальное приближение: (10.0; -8.0)

x_0	y_0	ε	$iters$	$funcs$	x	y	$f(x,y)$
10	-8	1.E-03	3	97	0.899703	0.899193	1.008553E-02
10	-8	1.E-04	3	132	0.989929	0.989878	1.016822E-04
10	-8	1.E-05	3	166	0.99899	0.998985	1.022694E-06
10	-8	1.E-06	3	196	0.999999	0.999999	1.215531E-12
10	-8	1.E-07	3	234	1	1	1.938488E-14

i	x_i	y_i	$f(x_i, y_i)$	s	λ	$ x_i - x_{i-1} $ $ y_i - y_{i-1} $ $ f_i - f_{i-1} $	$angle$	$grad$	η
1	1.6E+00	3.3E-01	1.7E+02	[[3.6e+03 -3.6e+03]]	-2.31E-03	['9.3e+00', '9.3e+00', '4.0e+04']	56	[[3.6e+03 -3.6e+03]]	[[1.0e+00 0.0e+00] [0.0e+00 1.0e+00]]
2	9.0E-01	9.0E-01	1.0E-02	[[7.3e-01 -5.7e-01]]	-1.0E+00	['7.3e-01', '5.7e-01', '1.7e+02']	83	[[2.6e+02 -2.6e+02]]	[[5.0e-01 5.0e-01] [5.0e-01 5.0e-01]]
3	9.0E-01	9.0E-01	1.0E-02	[[1.0e-02 -1.0e-02]]	-1.9E-03	['2.0e-05', '1.9e-05', '1.4e-07']	89	[[1.0e-02 -1.0e-02]]	[[1.0e+00 0.0e+00] [0.0e+00 1.0e+00]]

Начальное приближение: (15; -5)

x_0	y_0	ε	$iters$	$funcs$	x	y	$f(x, y)$
15	-5	1.E-03	3	100	0.915468	0.914897	7.178313E-03
15	-5	1.E-04	3	132	1.000005	1.000004	5.188568E-11
15	-5	1.E-05	3	166	1.000001	1.000001	8.990569E-13
15	-5	1.E-06	3	196	1	1	9.567345E-14
15	-5	1.E-07	3	234	1	1	3.887754E-15

i	x_i	y_i	$f(x_i, y_i)$	s	λ	$ x_i - x_{i-1} $ $ y_i - y_{i-1} $ $ f_i - f_{i-1} $	$angle$	$grad$	η
1	5.68E+00	4.25E+00	2.27E+02	[[4.0e+03 -4.0e+03]]	-2.31E-03	['9.3e+00', '9.3e+00', '4.0e+04']	82	[[4.0e+03 -4.0e+03]]	[[1.0e+00 0.0e+00] [0.0e+00 1.0e+00]]
2	9.16E-01	9.14E-01	7.60E-03	[[4.8e+00 3.3e+00]]	-9.99E-01	['4.8e+00', '3.3e+00', '2.3e+02']	10	[[3.0e+02 -2.9e+02]]	[[5.0e-01 5.0e-01] [5.0e-01 5.0e-01]]
3	9.15E-01	9.15E-01	7.18E-03	[[4.3e-01 -4.0e-01]]	-2.31E-03	['1.0e-03', '9.2e-04', '4.2e-04']	88	[[4.3e-01 -4.0e-01]]	[[1.0e+00 0.0e+00] [0.0e+00 1.0e+00]]

Функция Розенброка:

Начальное приближение: (10.0; -8.0)

x_0	y_0	ε	$iters$	$funcs$	x	y	$f(x, y)$
10	-8	1.E-03	12	354	-8.3E+23	5689400	4.674335E+97
10	-8	1.E-04	49	2108	1.005423	1.010918	2.959282E-05
10	-8	1.E-05	19	1036	0.999958	0.999916	1.767761E-09
10	-8	1.E-06	21	1386	0.999999	0.999998	1.083482E-12
10	-8	1.E-07	21	1627	1	1	8.200434E-16

i	x_i	y_i	$f(x_i, y_i)$	s	λ	$ x_i - x_{i-1} $ $ y_i - y_{i-1} $ $ f_i - f_{i-1} $	$angle$	$grad$	η
1	-5.30E+01	-4.85E+00	7.94E+08	[[4.3e+05 -2.2e+04]]	-1.46E-04	['6.3e+01', '3.2e+00', '7.9e+08']	172	[[4.3e+05 -2.2e+04]]	[[1.0e+00 0.0e+00] [0.0e+00 1.0e+00]]
2	-7.64E+01	3.56E+03	5.17E+08	[[1.7e+02 -2.5e+04]]	-1.40E-01	['2.3e+01', '3.6e+03', '2.8e+08']	179	[[-6.0e+07 -5.6e+05]]	[[8.2e-05 -9.0e-03] [-9.0e-03 1.0e+00]]

3	1.01E+04	3.63E+03	1.02E+18	[[-6.9e+07 -4.5e+05]]	-1.46E-04	['1.0e+04', '6.6e+01', '1.0e+18']	161	[[-6.9e+07 -4.5e+05]]	[[1.0e+00 0.0e+00] [0.0e+00 1.0e+00]]
4	6.56E+02	4.29E+05	1.07E+08	[[1.0e+04 -4.6e+05]]	-9.30E-01	['9.4e+03', '4.3e+05', '1.0e+18']	179	[[4.1e+14 -2.0e+10]]	[[2.5e-09 5.0e-05] [5.0e-05 1.0e+00]]
5	-3.88E+04	4.30E+05	2.27E+20	[[2.7e+08 -2.1e+05]]	-1.46E-04	['3.9e+04', '3.0e+01', '2.3e+20']	95	[[2.7e+08 -2.1e+05]]	[[1.0e+00 0.0e+00] [0.0e+00 1.0e+00]]
6	-8.00E+02	6.31E+05	8.16E+09	[[-3.9e+04 -2.1e+05]]	-9.63E-01	['3.8e+04', '2.0e+05', '2.3e+20']	169	[[-2.3e+16 -3.0e+11]]	[[1.7e-10 -1.3e-05] [-1.3e-05 1.0e+00]]
7	4.21E+05	6.32E+05	3.14E+24	[[-2.9e+09 -1.8e+06]]	-1.46E-04	['4.2e+05', '2.6e+02', '3.1e+24']	124	[[-2.9e+09 -1.8e+06]]	[[1.0e+00 0.0e+00] [0.0e+00 1.0e+00]]
8	-1.59E+03	2.47E+06	3.12E+11	[[4.2e+05 -1.8e+06]]	-1.01E+00	['4.2e+05', '1.8e+06', '3.1e+24']	167	[[3.0e+19 -3.5e+13]]	[[1.4e-12 1.2e-06] [1.2e-06 1.0e+00]]
9	5.17E+06	2.47E+06	7.16E+28	[[-3.5e+10 -1.1e+07]]	-1.46E-04	['5.2e+06', '1.6e+03', '7.2e+28']	155	[[-3.5e+10 -1.1e+07]]	[[1.0e+00 0.0e+00] [0.0e+00 1.0e+00]]
10	-1.83E+03	5.62E+06	5.11E+14	[[1.8e+07 -1.1e+07]]	-2.82E-01	['5.2e+06', '3.2e+06', '7.2e+28']	121	[[5.5e+22 -2.6e+16]]	[[2.3e-13 4.8e-07] [4.8e-07 1.0e+00]]
11	2.42E+08	5.69E+06	3.43E+35	[[1.7e+12 4.5e+08]]	1.46E-04	['2.4e+08', '6.6e+04', '3.4e+35']	1	[[1.7e+12 4.5e+08]]	[[1.0e+00 0.0e+00] [0.0e+00 1.0e+00]]
12	-8.27E+23	5.69E+06	4.67E+97	[[5.7e+27 0.0e+00]]	-1.46E-04	['8.3e+23', '0.0e+00', '4.7e+97']	180	[[5.7e+27 0.0e+00]]	[[1.0e+00 0.0e+00] [0.0e+00 1.0e+00]]

Начальное приближение: (15; -5)

x_0	y_0	ε	$iters$	$funcs$	x	y	$f(x, y)$
15	-5	1.E-03	6	128	5.82E+35	33009321	1.145726E+145
15	-5	1.E-04	59	2534	0.997801	0.995589	4.865221E-06
15	-5	1.E-05	17	922	0.999978	0.999956	4.836685E-10
15	-5	1.E-06	20	1333	0.999997	0.999998	1.023860E-09
15	-5	1.E-07	23	1782	1	1	1.085310E-15

i	x_i	y_i	$f(x_i, y_i)$	s	λ	$ x_i - x_{i-1} $ $ y_i - y_{i-1} $ $ f_i - f_{i-1} $	$angle$	$grad$	η
1	-1.86E+02	1.71E+00	1.21E+11	[[1.4e+06 -4.6e+04]]	-1.46E-04	['2.0e+02', '6.7e+00', '1.2e+11']	179	[[1.4e+06 -4.6e+04]]	[[1.0e+00 0.0e+00] [0.0e+00 1.0e+00]]
2	-1.54E+02	2.36E+04	6.30E+04	[[-6.9e+01 -5.0e+04]]	-4.75E-01	['3.3e+01', '2.4e+04', '1.2e+11']	180	[[-2.6e+09 -6.9e+06]]	[[7.2e-06 -2.7e-03] [-2.7e-03 1.0e+00]]
3	2.51E+01	2.36E+04	5.28E+10	[[1.2e+06 4.0e+03]]	1.46E-04	['1.8e+02', '5.8e-01', '5.3e+10']	90	[[1.2e+06 4.0e+03]]	[[1.0e+00 0.0e+00] [0.0e+00 1.0e+00]]
4	-3.36E+04	2.43E+04	1.28E+20	[[-2.3e+08 4.6e+06]]	1.46E-04	['3.4e+04', '6.7e+02', '1.3e+20']	35	[[-2.3e+08 4.6e+06]]	[[1.0e+00 0.0e+00] [0.0e+00 1.0e+00]]
5	2.22E+12	3.30E+07	2.42E+51	[[-1.5e+16 -2.3e+11]]	-1.46E-04	['2.2e+12', '3.3e+07', '2.4e+51']	180	[[-1.5e+16 -2.3e+11]]	[[1.0e+00 0.0e+00] [0.0e+00 1.0e+00]]
6	5.82E+35	3.30E+07	1.15E+14 5	[[4.0e+39 0.0e+00]]	1.46E-04	['5.8e+35', '0.0e+00', '1.1e+145']	0	[[4.0e+39 0.0e+00]]	[[1.0e+00 0.0e+00] [0.0e+00 1.0e+00]]

Функция по варианту:

Начальное приближение: (10.0; -8.0)

x_0	y_0	ε	$iters$	$funcs$	x	y	$f(x, y)$
10	-8	1.E-03	1	16	10	-8	2.458204E-39
10	-8	1.E-04	1	16	10	-8	2.458204E-39
10	-8	1.E-05	1	16	10	-8	2.458204E-39
10	-8	1.E-06	1	16	10	-8	2.458204E-39
10	-8	1.E-07	1	16	10	-8	2.458204E-39

i	x_i	y_i	$f(x_i, y_i)$	s	λ	$ x_i - x_{i-1} $ $ y_i - y_{i-1} $ $ f_i - f_{i-1} $	$angle$	$grad$	η
1	1.00E+01	-8.00E+00	2.46E-39	[[-4.4e-38 4.9e-39]]	6.18E-04	['0.0e+00', '0.0e+00', '0.0e+00']	148	[[-4.4e-38 4.9e-39]]	[[1.0e+00 0.0e+00] [0.0e+00 1.0e+00]]

Начальное приближение: (15; -5)

x_0	y_0	ε	$iters$	$funcs$	x	y	$f(x, y)$
15	-5	1.E-03	1	16	15	-5	3.039405E-84
15	-5	1.E-04	1	16	15	-5	3.039405E-84
15	-5	1.E-05	1	16	15	-5	3.039405E-84
15	-5	1.E-06	1	16	15	-5	3.039405E-84
15	-5	1.E-07	1	16	15	-5	3.039405E-84

i	x_i	y_i	$f(x_i, y_i)$	s	λ	$ x_i - x_{i-1} $ $ y_i - y_{i-1} $ $ f_i - f_{i-1} $	$angle$	$grad$	η
1	1.50E+01	-5.00E+00	3.04E-84	[[-7.2e-83 4.3e-83]]	6.18E-04	['0.0e+00', '0.0e+00', '0.0e+00']	168	[[-7.2e-83 4.3e-83]]	[[1.0e+00 0.0e+00] [0.0e+00 1.0e+00]]

Выводы

Так как метод Бройдена использует информацию о вторых производных функции, этот метод эффективнее решает задачу с квадратичными функциями чем метод Розенброка. Точность решения зависит от начального приближения – иногда методы могут застревать в локальных минимумах.

Код программы:

broyden.py

```
from config import *
from math import exp, acos, sqrt
import numpy as np
import csv

funcs_count: int = 0

eps_range = [1e-3, 1e-4, 1e-5, 1e-6, 1e-7]

def test_func(x: np.array) -> float:
    global funcs_count
    funcs_count += 1
    return A1 * exp(- ((x[0, 0] - a1)/(b1))**2 - ((x[0, 1] - c1)/(d1))**2) + A2 *
exp(- ((x[0, 0] - a2)/(b2))**2 - ((x[0, 1] - c2)/(d2))**2)

def quad_func(x: np.array) -> float:
    global funcs_count
    funcs_count += 1
    return 100 * (x[0, 1] - x[0, 0])**2 + (1 - x[0, 0])**2

def rosen_func(x: np.array) -> float:
    global funcs_count
    funcs_count += 1
    return 100 * (x[0, 1] - x[0, 0]**2)**2 + (1 - x[0, 0])**2

def min_interval(begin: float, func: callable, delta: float, x: np.array, s: np.array) -> tuple[float, float]:
    """
    Finds an interval for search the minimum of a function

    :param begin: start point for searching
    :param func: callable function
    :param delta: precision for first iteration

    :return: void
    """

    iters = h = 0
    prev = curr = next = begin
    if (func(x + begin * s) > func(x + (begin + delta) * s)):
        next = begin + delta
        h = delta
    if (func(x + begin * s) > func(x + (begin - delta) * s)):
        next = begin - delta
        h = -delta
```

```

if (h != 0):
    while (func(x + curr * s) > func(x + next * s)):
        iters += 1
        prev = curr
        curr = next

        h *= 2
        next = curr + h

    return prev, next
else:
    return begin - delta, begin + delta

```

```

def golden_section(begin: int, end: int, func: callable, eps: float, x: np.array,
s: np.array) -> float:
    """

```

```

    Finds an interval that contains the minimum of a function
    Using a Golden Section Algorithm

```

```

:param begin: begin of initial interval
:param end: end of initial interval
:param func: callable function (x, y)
:param eps: precision

```

```

:return: void
"""

```

```

golden_number = 0.381966011

```

```

iters = 0

```

```

left = begin + golden_number * (end - begin)
right = end - golden_number * (end - begin)

```

```

value_left = func(x + left * s)
value_right = func(x + right * s)

```

```

while (abs(end - begin) > eps):
    iters += 1

```

```

    if (value_left < value_right):
        end = right
        right = left
        value_right = value_left
        left = begin + golden_number*(end - begin)
        value_left = func(x + left * s)

```

```

    else:
        begin = left
        left = right

```

```

        value_left = value_right
        right = end - golden_number*(end - begin)
        value_right = func(x + right * s)

    return (begin + end) / 2

def gradient(x: np.array, eps: float, func: callable) -> np.array:

    x = np.asmatrix(x)

    func_value = func(x)

    result = np.array([0, 0], dtype=float)
    _x = x

    for i in range(2):
        _x[0, i] += eps
        result[i] = (func(_x) - func_value) / eps
        _x[0, i] -= eps

    return result

def angle(x: np.array, s: np.array):
    return acos((x[0,0]*s[0,0] + x[0,1]*s[0,1]) / (sqrt(x[0,0]**2 + x[0,1]**2) *
sqrt(s[0,0]**2 + s[0,1]**2))) * 180 / 3.1415926535

def broyden(x: np.array, eps: float, target: callable):
    nu = np.matrix([[1, 0], [0, 1]], dtype = float)

    grad = gradient(x, eps, target)

    global funcs_count
    funcs_count = 0
    iters: int = 0
    deltaF: float = eps
    deltaX: bool = True

    with open(f'broyden/t2/{target.__name__}_{eps:1.1e}.csv', 'w', newline='') as
csvfile:
        spamwriter = csv.writer(csvfile, delimiter=';', quotechar='|', quot-
ing=csv.QUOTE_MINIMAL)

        while (deltaF >= eps and deltaX == True):

            iters += 1
            x_prev = x

            if (iters % 2 != 0):

```



```

        nu = np.matrix([[1, 0], [0, 1]], dtype = float)

    if (np.linalg.det(nu) <= 0):
        nu = np.matrix([[1, 0], [0, 1]], dtype = float)

    nugrad = np.matmul(nu, grad)

    begin, end = min_interval(0, target, eps, x, nugrad)
    lambd: float = golden_section(begin, end, target, eps, x, nugrad)

    dx = lambd * nugrad
    x = x + dx

    spamwriter.writerow([iters,
                          np.asmatrix(x)[0, 0],
                          np.asmatrix(x)[0, 1],
                          target(np.asmatrix(x)),
                          nugrad,
                          lambd,
                          [
                              f'{abs(np.asmatrix(x)[0, 0] - np.asma-
trix(x_prev)[0, 0]):1.1e}',
                              f'{abs(np.asmatrix(x)[0, 1] - np.asma-
trix(x_prev)[0, 1]):1.1e}',
                              f'{abs(target(np.asmatrix(x)) - target(np.asma-
trix(x_prev))):1.1e}',
                          ],
                          angle(np.asmatrix(x), nugrad),
                          np.asmatrix(grad),
                          nu])

    _grad = grad
    grad = gradient(x, eps, target)

    dgrad = grad - _grad

    tmp1 = dx - np.matmul(nu, dgrad)
    tmp2 = tmp1.transpose()

    tmp3 = np.matmul(tmp2, tmp1)

    tmp4 = tmp1 * np.asmatrix(dgrad).transpose()

    if (tmp4 == 0):
        nu = np.matrix([[1, 0], [0, 1]], dtype = float)
    else:
        dnu = tmp3 / tmp4
        nu = nu + dnu

    for i in range(2):
        if (abs(np.asmatrix(x)[0, i] - np.asmatrix(x_prev)[0, i]) < eps):

```

```

        deltaX = False
        break

    deltaF = abs(target(np.asmatrix(x)) - target(np.asmatrix(x_prev)))

    return (x, iters, funcs_count)

def main():
    x = np.array([15, -5], dtype = float)
    targets = {test_func, quad_func, rosen_func}

    np.set_printoptions(suppress=True,
                        formatter={'float_kind': '{:1.1e}'.format})

    for target in targets:
        with open(f'broyden/t1/{target.__name__}.csv', 'w', newline='') as csvfile:
            spamwriter = csv.writer(csvfile, delimiter=';', quotechar='|', quoting=csv.QUOTE_MINIMAL)
            for eps in eps_range:
                res, iters, funcs = broyden(x, eps, target)
                spamwriter.writerow([x[0], x[1], eps, iters, funcs, res[0, 0],
res[0, 1], target(res)])

if __name__ == "__main__":
    main()

```

rosenbrock.py

```

from config import *
from math import exp, acos, sqrt
import numpy as np
import csv

funcs_count: int = 0

eps_range = [1e-3, 1e-4, 1e-5, 1e-6, 1e-7]

def test_func(x: np.array) -> float:
    global funcs_count
    funcs_count += 1
    return A1 * exp(- ((x[0, 0] - a1)/(b1))**2 - ((x[0, 1] - c1)/(d1))**2) + A2 *
exp(- ((x[0, 0] - a2)/(b2))**2 - ((x[0, 1] - c2)/(d2))**2)

def quad_func(x: np.array) -> float:
    global funcs_count
    funcs_count += 1
    return 100 * (x[0, 1] - x[0, 0])**2 + (1 - x[0, 0])**2

```

```

def rosen_func(x: np.array) -> float:
    global funcs_count
    funcs_count += 1
    return 100 * (x[0, 1] - x[0, 0]**2)**2 + (1 - x[0, 0])**2

def min_interval(begin: float, func: callable, delta: float, x: np.array, s: np.array) -> tuple[float, float]:
    """
    Finds an interval for search the minimum of a function

    :param begin: start point for searching
    :param func: callable function
    :param delta: precision for first iteration

    :return: void
    """

    iters = h = 0
    prev = curr = next = begin

    if (func(x + begin * s) > func(x + (begin + delta) * s)):
        next = begin + delta
        h = delta
    if (func(x + begin * s) > func(x + (begin - delta) * s)):
        next = begin - delta
        h = -delta

    if (h != 0):
        while (func(x + curr * s) > func(x + next * s)):
            iters += 1
            prev = curr
            curr = next

            h *= 2
            next = curr + h

        return prev, next
    else:
        return begin - delta, begin + delta

def golden_section(begin: int, end: int, func: callable, eps: float, x: np.array, s: np.array) -> float:
    """
    Finds an interval that contains the minimum of a function
    Using a Golden Section Algorithm

    :param begin: begin of initial interval

```

```
:param end: end of initial interval
:param func: callable function (x, y)
:param eps: precision
```

```
:return: void
"""
```

```
golden_number = 0.381966011
```

```
iters = 0
```

```
left = begin + golden_number * (end - begin)
right = end - golden_number * (end - begin)
```

```
value_left = func(x + left * s)
value_right = func(x + right * s)
```

```
while (abs(end - begin) > eps):
    iters += 1
```

```
    if (value_left < value_right):
        end = right
        right = left
        value_right = value_left
        left = begin + golden_number*(end - begin)
        value_left = func(x + left * s)
```

```
    else:
        begin = left
        left = right
        value_left = value_right
        right = end - golden_number*(end - begin)
        value_right = func(x + right * s)
```

```
return (begin + end) / 2
```

```
def angle(x: np.array, s: np.array):
```

```
    return acos((x[0,0]*s[0,0] + x[0,1]*s[0,1]) / (sqrt(x[0,0]**2 + x[0,1]**2) *
sqrt(s[0,0]**2 + s[0,1]**2))) * 180 / 3.1415926535
```

```
def rosenbrock(x: np.array, eps: float, target: callable):
```

```
    s = np.matrix([[1, 0], [0, 1]], dtype = float)
    a = np.matrix([[0, 0], [0, 0]], dtype = float)
```

```
    global funcs_count
```

```
    funcs_count = 0
```

```
    iters: int = 0
```

```
    deltaF: float = eps
```

```
    deltaX: bool = True
```

```

    with open(f'rosenbrock/t2/{target.__name__}_{eps:1.1e}.csv', 'w', newline='')
as csvfile:
    spamwriter = csv.writer(csvfile, delimiter=';', quotechar='|', quoting=csv.QUOTE_MINIMAL)

    while (deltaF >= eps and deltaX == True):

        iters += 1
        x_prev = np.asmatrix(x)

        begin, end = min_interval(0, target, eps, x, s[0])
        lambda1: float = golden_section(begin, end, target, eps, x, s[0])

        x = x + s[0] * lambda1

        spamwriter.writerow([iters, x[0, 0], x[0, 1], target(x), s[0, 0], s[0,
1], lambda1, abs(x[0, 0] - x_prev[0, 0]), abs(x[0, 1] - x_prev[0, 1]), abs(target(x) - target(x_prev)), angle(x, s[0])])

        begin, end = min_interval(0, target, eps, x, s[1])
        lambda2: float = golden_section(begin, end, target, eps, x, s[1])

        x = x + s[1] * lambda2

        spamwriter.writerow([iters, x[0, 0], x[0, 1], target(x), s[1, 0], s[1,
1], lambda2, abs(x[0, 0] - x_prev[0, 0]), abs(x[0, 1] - x_prev[0, 1]), abs(target(x) - target(x_prev)), angle(x, s[1])])

        a[0] = lambda1 * s[0] + lambda2 * s[1]

        if (abs(lambda1) < abs(lambda2)):
            a[1] = lambda1 * s[1]
        else:
            a[1] = lambda2 * s[1]

        s[0] = a[0] / np.linalg.norm(a[0])

        b = a[1] - (a[1] * s[0].transpose()) * s[0]

        s[1] = b / np.linalg.norm(b)

        for i in range(2):
            if (abs(x[0, i] - x_prev[0, i]) < eps):
                deltaX = False
                break

        deltaF = abs(target(x) - target(x_prev))

    return(x, iters, funcs_count)

```

```

def main():
    x = np.array([-2500, 6000], dtype = float)
    targets = {test_func, quad_func, rosen_func}

    for target in targets:
        with open(f'rosenbrock/t1/{target.__name__}.csv', 'w', newline='') as
csvfile:
            spamwriter = csv.writer(csvfile, delimiter=';', quotechar='|', quot-
ing=csv.QUOTE_MINIMAL)
            for eps in eps_range:
                res, iters, funcs = rosenbrock(x, eps, target)
                spamwriter.writerow([x[0], x[1], eps, iters, funcs, res[0, 0],
res[0, 1], target(res)])

if __name__ == "__main__":
    main()

```